0

[Week 12]

chelseey · 3일 전
통계 수정 삭제

Euron

트랜스포머

: 순차적 처리(순환 신경망)나 반복 연결 없이도
긴 시퀀스를 빠르고 효율적으로 다룰 수 있는 구조

• 핵심 아이디어: 셀프-어텐션(Self-Attention)

- 모든 입력 토큰 쌍 간의 상호 관계를 한 번에 계산
- 토큰들 사이의 먼 거리 의존성(dependency)도 직접 모델링 가능
- 병렬 연산이 가능해 순환 신경망보다 학습/추론 속도가 훨씬 빠름

• 학습 방식

- 오토인코딩(Auto-Encoding, 양방향 인코더)
  문장 일부를 빈칸 토큰으로 만든 뒤, 해당 토큰을 예측하도록 학습

- 자가회귀(Auto-Regressive, 단방향 디코더)
  주어진 앞쪽(또는 앞뒤) 토큰을 보고 다음 토큰을 순차적으로 예측

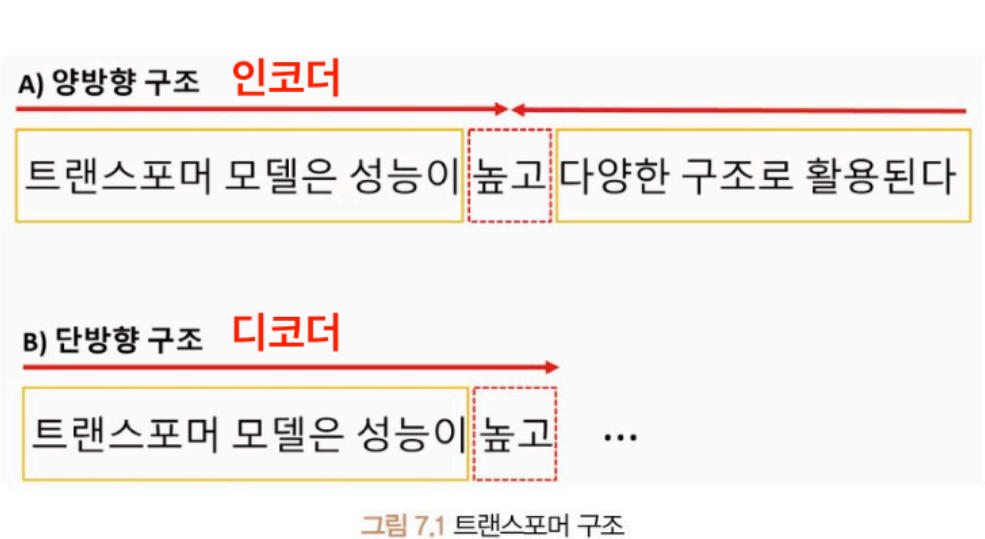
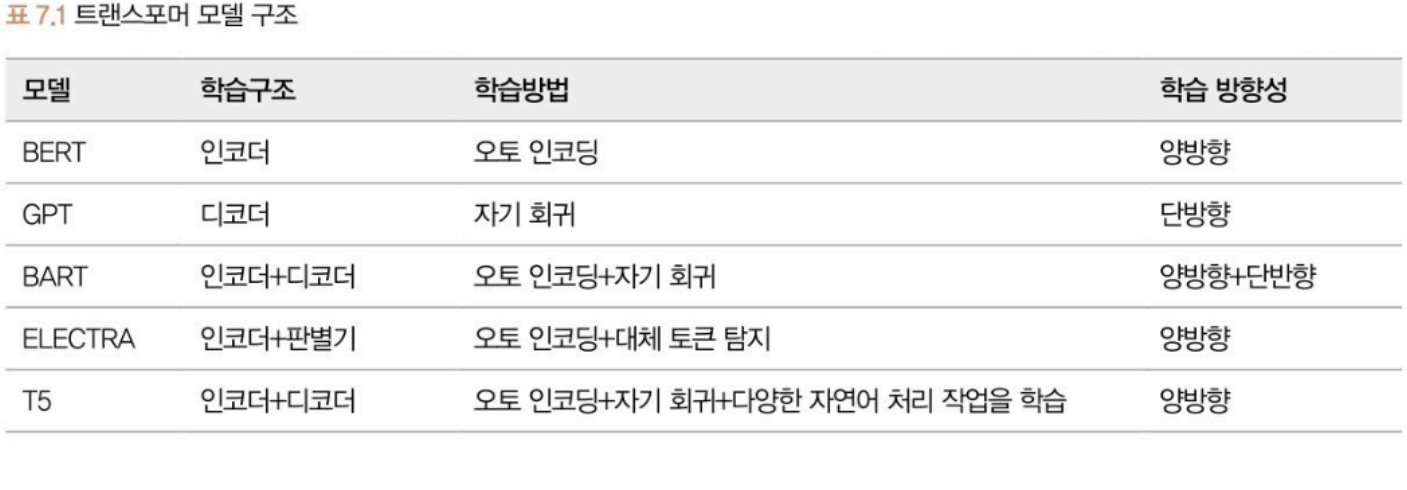


그림 7.1 트랜스포머 구조

→ 인코더는 주로 이해(분류·추론), 디코더는 주로 생성(번역·생성) 작업에 활용

표 7.1 트랜스포머 모델 구조



모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단반향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

Transformer

- 순환(RNN)·합성곱(CNN) 대신 **어텐션 메커니즘**만으로
  시퀀스 전체를 병렬 처리
- 긴 문장도 속도 저하 없이 학습·추론 가능
- 기계 번역, 요약, 챗봇 등 다양한 자연어 처리 작업에서 뛰어난 성능

• 인코더–디코더 구조

- 소스(Source) 시퀀스 → 인코더 → 잠재 표현(고차원 벡터)
- 인코더 출력 + 디코더 입력(타겟 시퀀스 일부) → 디코더 → 다음 토큰 예측

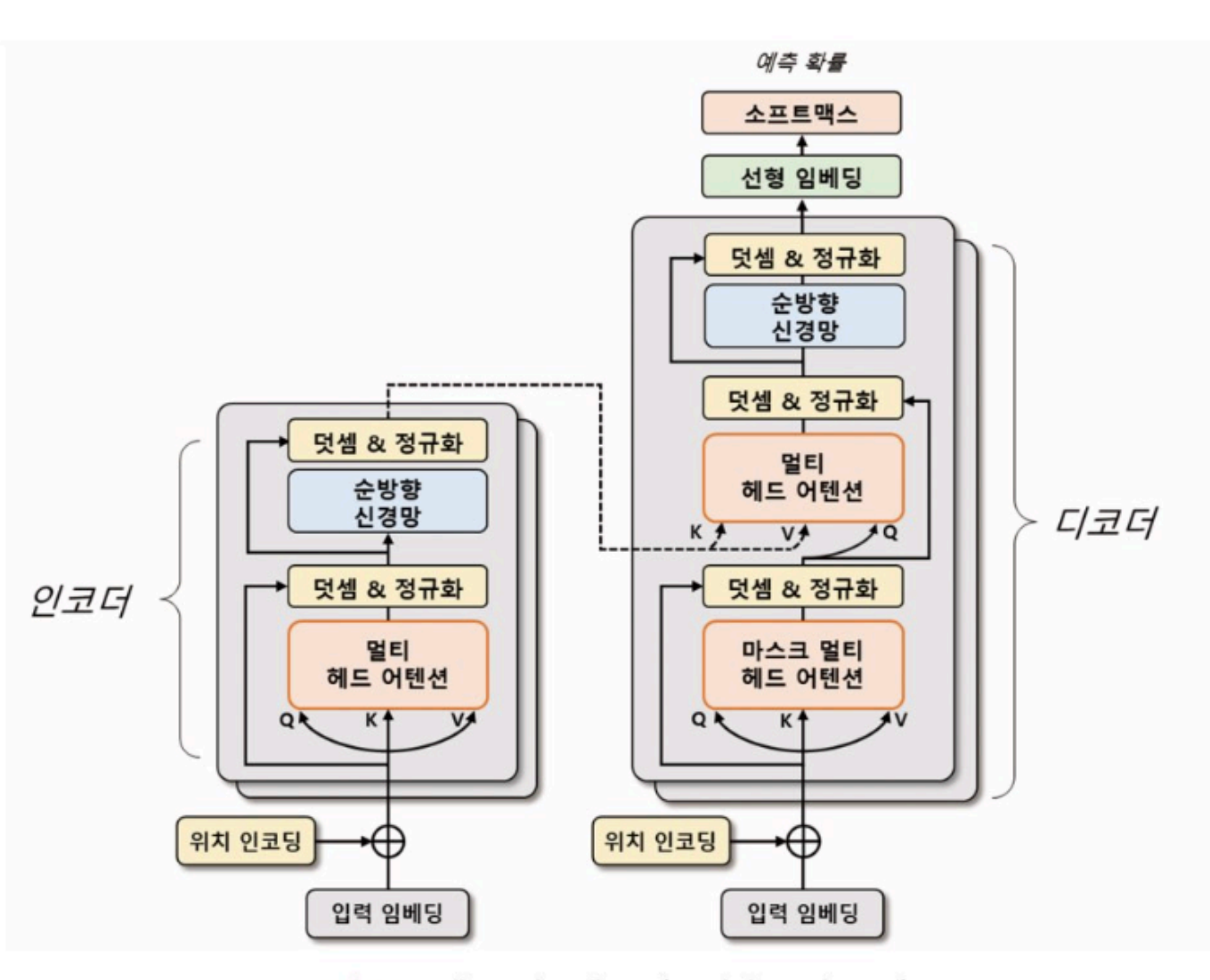


그림 7.2 트랜스포머 모델 구조(Q: 쿼리, K: 키, V: 값)

인코더(Encoder)

여러 개의 Transformer Block으로 구성

각 블록 내부 :

- 멀티-헤드 셀프 어텐션(Multi-Head Self-Attention)
  : 입력 시퀀스의 각 위치에 대해 Query·Key·Value를 생성.
  토큰 간 유사도(어텐션 가중치)를 계산하여 값(Value) 벡터를 가중합.

- 순방향 FFN & 레지듀얼 연결
  : 어텐션 출력에 선형층·비선형 활성화(ReLU) 적용.
  레지듀얼(잔차) 연결 및 층 정규화(LayerNorm)

- 위치 인코딩(Positional Encoding)
  : 셀프 어텐션 자체는 토큰 순서를 구분하지 못하기 때문에,
  입력 임베딩에 순서 정보를 추가 (소스 위치 정보 인코딩)

디코더(Decoder)

인코더와 마찬가지로 여러 Transformer Block으로 구성

- 위치 인코딩(Positional Encoding)
  디코더 입력 토큰 임베딩에 순서 정보를 부여.
  (타깃 위치 정보 인코딩)

각 블록 내부 :

- 마스크드 멀티-헤드 셀프 어텐션
  : 미래 토큰 정보가 보이지 않도록 마스킹하여 자동회귀 생성.

- 인코더-디코더 어텐션
  : 디코더의 Query가 인코더 출력(Key·Value)에 어텐션을 수행하여
  소스 문맥 반영.

- 순방향 FFN & 레지듀얼 연결

최종 출력 처리

- 디코더 최종 레이어 출력 → 선형 임베딩 차원으로 투영 → 소프트맥스
- 생성된 확률 분포에서 다음 토큰 샘플링 또는 최대값 선택

입력 임베딩과 위치 인코딩

Transformer는 RNN/CNN과 달리 순차 처리 없이 병렬로 입력 시퀀스를 처리하므로, 단어의 위치(순서) 정보가 모델에 제공되지 않음.

→ 각 토큰 임베딩에 '위치 정보'를 더해 주어야 순서 관계를 학습할 수 있음.

위치 인코딩(Positional Encoding) 방식

각 위치 `pos` 와 임베딩 차원 인덱스 `i` 를 입력으로,
사인·코사인 함수를 사용해 벡터를 계산

$PE(pos,2i) = \sin(pos/10000^{2i/d_{model}})$
$PE(pos,2i+1) = \cos(pos/10000^{2i/d_{model}})$

짝수 인덱스 차원에는 sin, 홀수 인덱스 차원에는 cos를 적용
분모의 스케일링 인자 $10000^{2i/d_{model}}$를 통해 다양한 주기의 주기적 신호 생성

초기화 단계 :

- `max_len` (최대 시퀀스 길이)와 `d_model` (임베딩 차원)을 입력 받아, 크기 `[max_len, 1, d_model]` 의 텐서 `pe` 를 미리 계산.

- `torch.sin` , `torch.cos` 로 각 위치·차원별 값을 채워 두고 `register_buffer("pe", pe)` 로 학습 대상이 아니게 등록.

forward 단계 :

- 입력 임베딩 x (크기 `[seq_len, batch, d_model]`)에,
  미리 계산된 `pe[:seq_len]` 을 더함

- 그 뒤 Dropout을 적용하여 순서 정보가 결합된 최종 입력을 생성

특수 토큰

특수 토큰의 역할

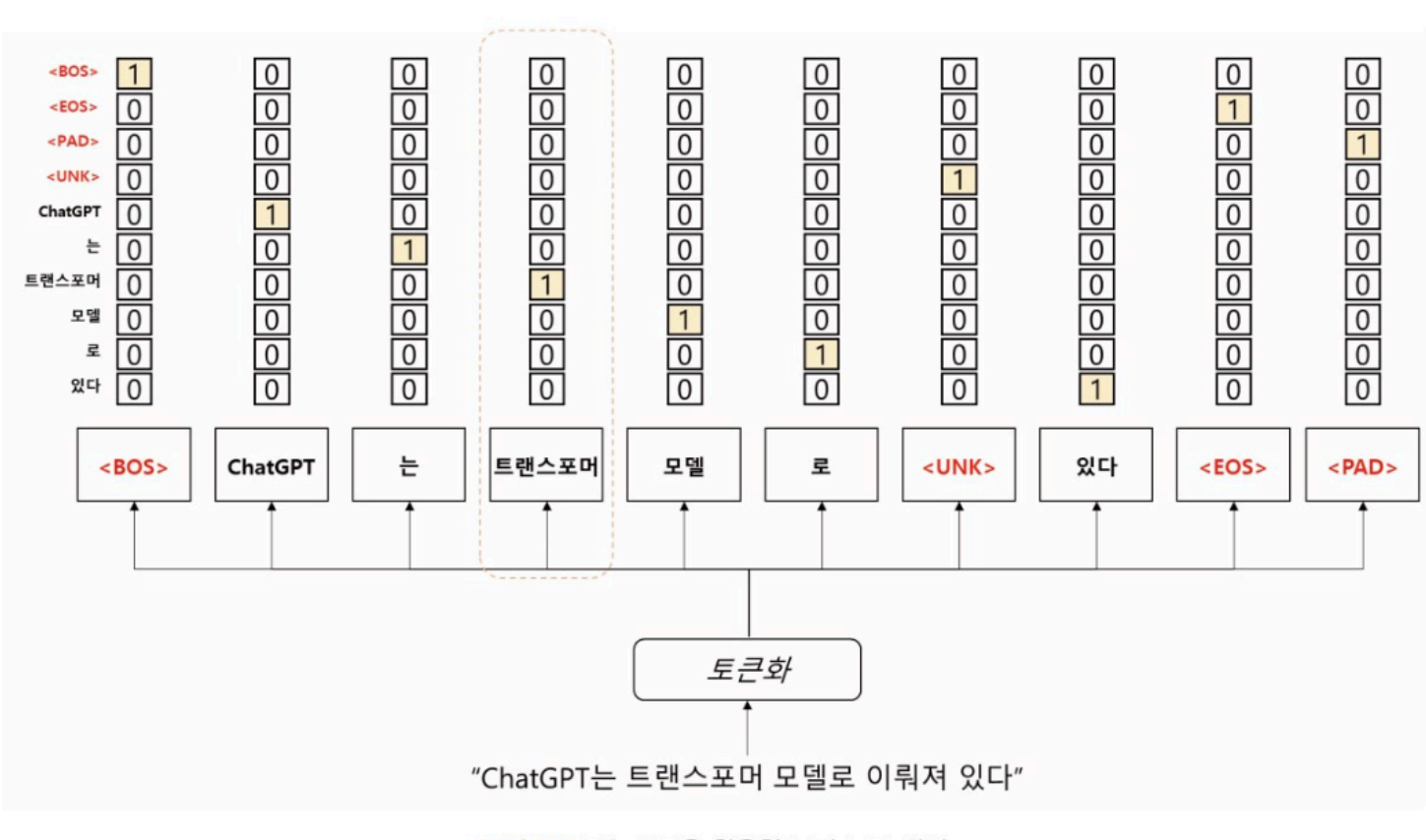


그림 7.3 특수 토큰을 활용한 문장 토큰 배열

문장 구조 표시 :

- `<BOS>` : 문장의 시작 표시
- `<EOS>` : 문장의 끝 표시

미등록·패딩 처리 :

- `<UNK>` : 어휘 사전에 없는 알 수 없는 토큰 처리
- `<PAD>` : 짧은 문장을 일정 길이로 맞추기 위해 뒤에 채워 넣는 토큰

마스킹(Masking) :

- 디코더의 자동 회귀 방식 시,
  미래 토큰을 가리기 위해 마스크 토큰처럼 사용 가능

입력 임베딩 과정

- 토큰화 & 인덱싱
  입력 문장을 어휘 사전(vocabulary) 토큰 시퀀스로 변환
  각 토큰에 0부터 V–1까지 정수 인덱스 부여

- 원-핫(one-hot) 표현
  어휘 사전 크기 V인 벡터에서 해당 토큰 인덱스 위치만 1, 나머지 0

ex. "트랜스포머" 인덱스 i → `[0,…,0,1,0,…,0]` (길이 V)

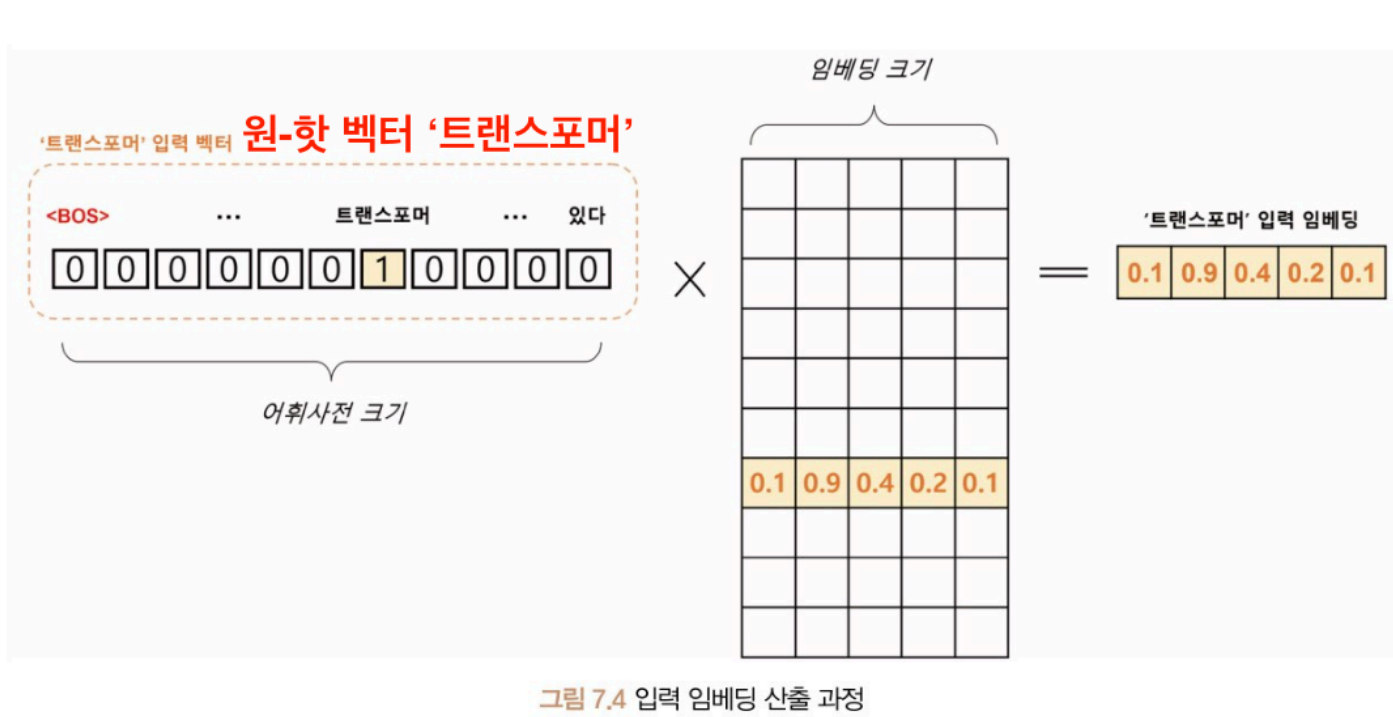


그림 7.4 입력 임베딩 산출 과정

- 임베딩 룩업
  크기 `[V, d]` 임베딩 행렬 E 준비 (d는 임베딩 차원)
  원-핫 벡터 × E → 길이 d의 실수 벡터 (토큰 임베딩)
  전체 시퀀스 S토큰 → `[S, d]` 크기 텐서

- 모델 전 계층 공유
  얻은 임베딩 텐서는 Transformer의 인코더·디코더 전 블록에서
  동일하게 사용

트랜스포머 인코더

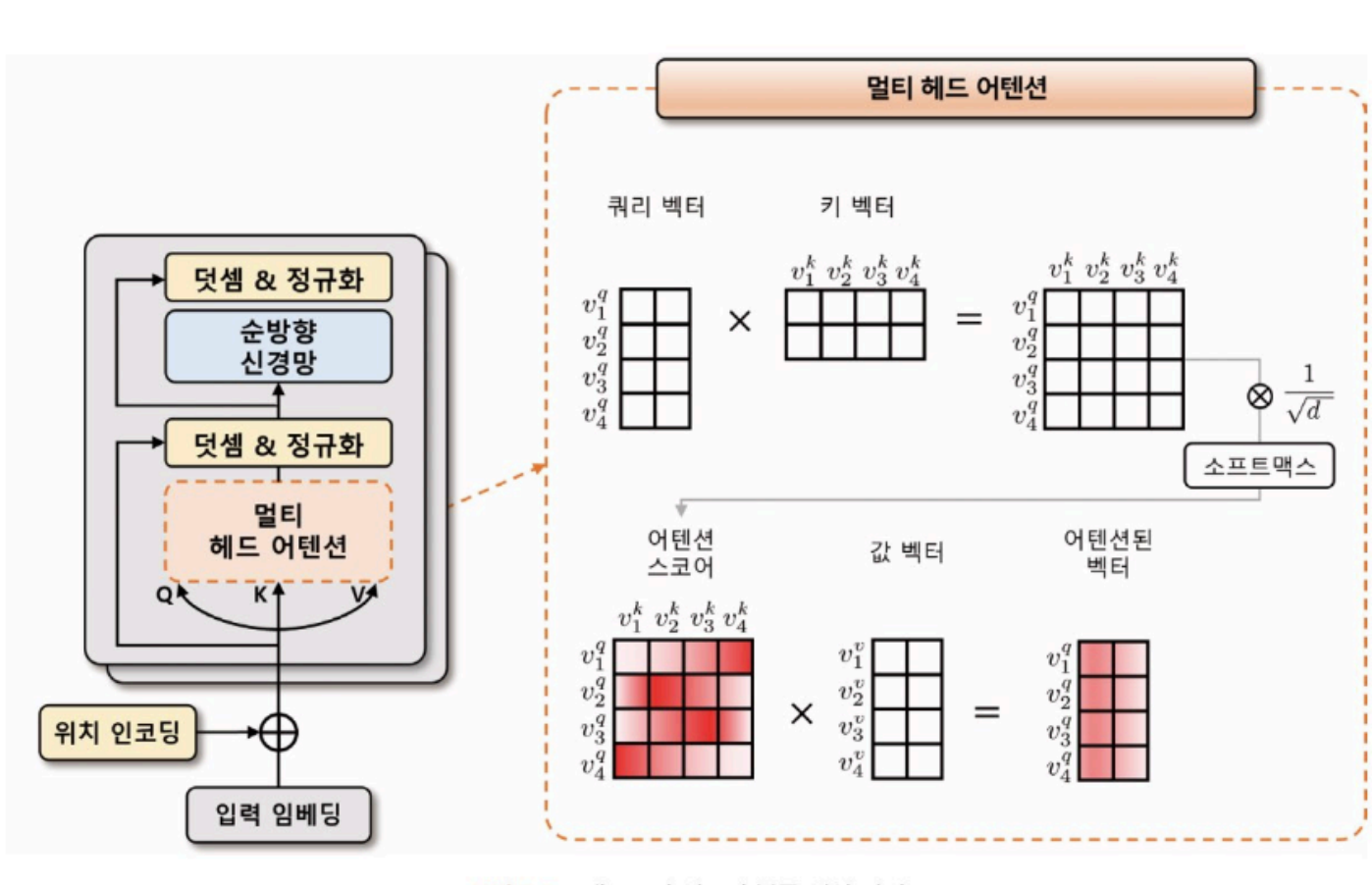


그림 7.5 트랜스포머 인코더 블록 연산 과정

입력 임베딩 + 위치 인코딩

- 소스 시퀀스의 토큰을 임베딩 벡터로 변환
- 단어의 순서 정보를 주기 위해 위치 인코딩(PE)을 더함

멀티-헤드 셀프 어텐션

- 임베딩된 입력을 세 개의 선형층으로 변환하여
  쿼리(Q) · 키(K) · 값(V) 벡터 생성

- 각 토큰의 Q·K 내적을 √d로 스케일링 후
  소프트맥스로 어텐션 스코어를 계산
  $score(v^i, v^k) = \text{softmax}\left(\frac{v^i \cdot v^k}{\sqrt{d}}\right)$

- 어텐션 스코어로 원본 값(Value) 벡터를 가중합 → 셀프 어텐션된 벡터 생성

- 이 과정을 여러 개(헤드) 동시 수행
  → 결과를 이어 붙여(concat) 하나의 벡터로 합침

인코더→디코더 정보 공유

최종 인코더 블록의 출력 벡터들을 디코더로 전달
→ 디코더의 인코더–디코더 멀티-헤드 어텐션에서 Key·Value로 사용
→ 양쪽이 키·값 벡터를 주고받아 문맥적 생성 및 번역 성능 강화

트랜스포머 디코더

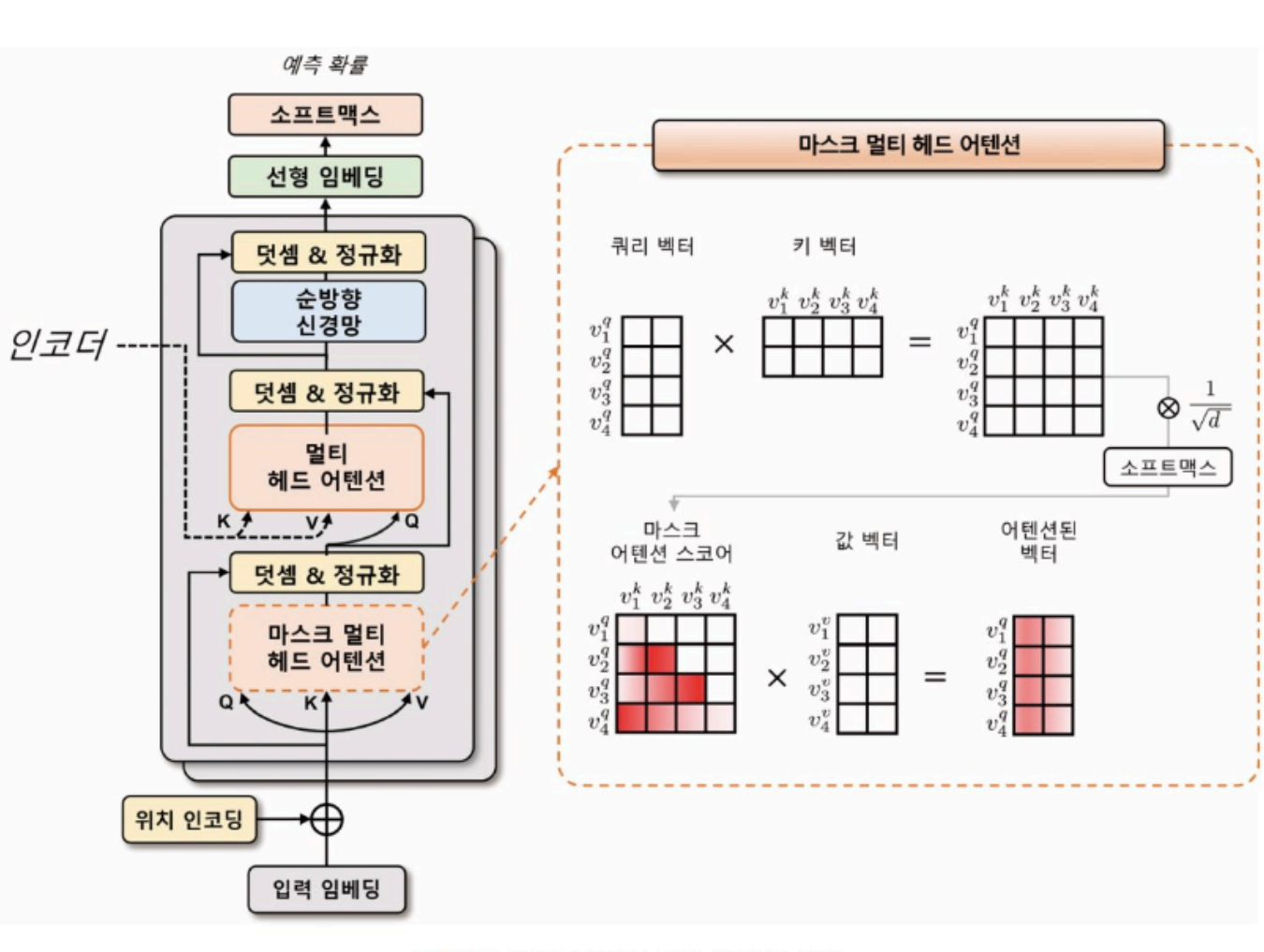


그림 7.6 트랜스포머 디코더 블록 연산 과정

입력 처리

- 디코더에도 위치 인코딩이 더해진 임베딩을 사용하여 시퀀스 순서 정보 반영

마스크드 멀티-헤드 셀프 어텐션 (Masked Self-Attention)

- 현재 위치 이후 토큰들이 보이지 않도록 음의 무한대(`-inf`) 마스크 적용.
- 어텐션 스코어 계산 시 마스크된 위치의 확률이 0에 가까워지며, 인과성(causality) 반영.

인코더–디코더 어텐션 (Encoder-Decoder Attention)

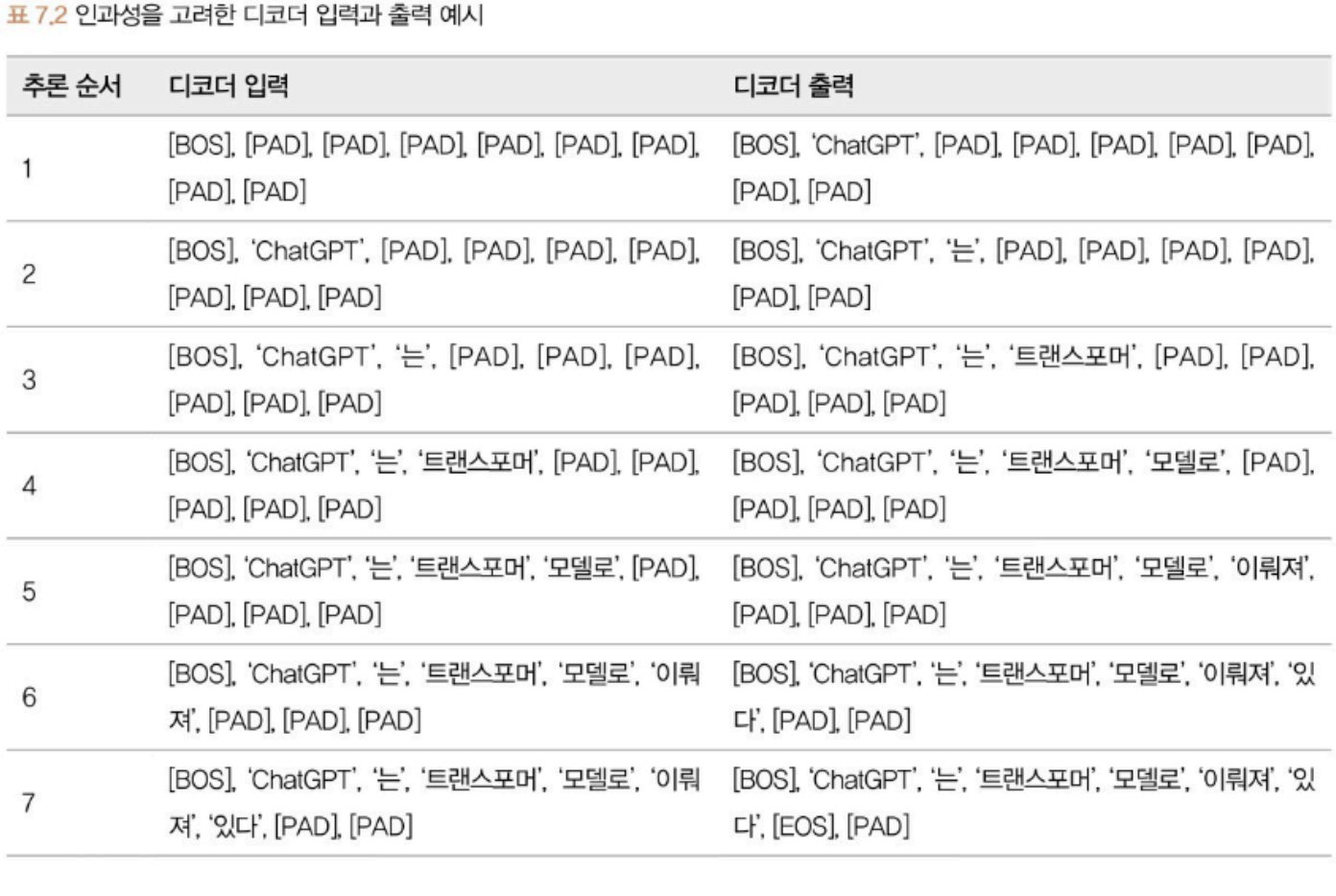
- 디코더의 Query가 디코더 입력(위치+임베딩)을, Key·Value는 인코더 최종 출력 벡터를 참조
- 소스(인코더) 정보와 타겟(디코더) 정보를 결합하여 문맥을 반영

Add & Norm → FFN → Add & Norm

- 마스크드 셀프 어텐션 → 잔차 연결 + 층 정규화
- 인코더–디코더 어텐션 → 잔차 연결 + 층 정규화
- 피드포워드 네트워크(선형→ReLU→선형 또는 1D 합성곱)
  → 잔차 연결 + 층 정규화

출력 생성

표 7.2 인과성을 고려한 디코더 입력과 출력 예시



추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD]

- 마지막 블록의 출력에 선형 투영 및 소프트맥스 적용
  → 각 위치마다 다음 토큰의 예측 확률 산출

- 예측된 토큰을 순차적으로 생성하며,
  빈 자리는 PAD 토큰으로 채워서 다음 단계 입력으로 사용

모델 실습

TokenEmbedding

: 어휘 사전 크기(Vocab size)와 임베딩 차원(emb_size)을 받아 `nn.Embedding(vocab_size, emb_size)` 로 토큰 인덱스를 벡터로 변환
벡터 크기를 √emb_size 만큼 스케일링

PositionalEncoding

: 최대 시퀀스 길이(`max_len`)와 임베딩 차원(`d_model`)을 받아
각 위치(pos)와 차원 인덱스(i)에 대해

- 짝수 차원: $PE(pos,2i) = sin(pos/10000^{2i/d_{model}})$
- 홀수 차원: $PE(pos,2i+1) = cos(pos/10000^{2i/d_{model}})$

미리 계산한 위치 인코딩(`pe`)을 등록버퍼로 저장
순전파 시 입력 임베딩에 `pe` 를 더하고 드롭아웃

Seq2SeqTransformer

초기화

- `src_tok_emb` (소스), `tgt_tok_emb` (타깃) : TokenEmbedding

- `positional_encoding` : PositionalEncoding

- `self.transformer = nn.Transformer(...)`

- `self.generator = nn.Linear(emb_size, tgt_vocab_size)`
  : 디코더 최종 출력을 어휘 크기만큼의 로짓(logit)으로 변환

순전파(forward)

- 소스(`src`)와 타깃(`trg`) 토큰 인덱스를 임베딩 → 위치 인코딩

```
src_emb = positional_encoding(src_tok_emb(src))
tgt_emb = positional_encoding(tgt_tok_emb(trg))
```

- `self.transformer` 에 소스/타깃 임베딩,
  마스크(mask)·패딩(padding) 마스크 전달

- 디코더 출력(outs)을 `self.generator` 에 통과시켜 최종 로짓 반환

인코더·디코더 분리

```
def encode(self, src, src_mask):
    return transformer.encoder(pos_enc(src_tok_emb(src)), src_mask)

def decode(self, tgt, memory, tgt_mask):
    return transformer.decoder(pos_enc(tgt_tok_emb(tgt)), memory, tgt_mask)
```

`encode` : 소스→임베딩+위치→인코더 블록 → 메모리
`decode` : 타깃→임베딩+위치 + 메모리 → 디코더 블록

어텐션 마스크(attention mask)

: 디코더 셀프-어텐션에서 현재 위치 이후의 토큰을 보지 못하게
(미래 정보 차단)하여 오토리그레시브 생성 인과성(causality)을 보장.

패딩 마스크(padding mask)

: 실제 토큰과 패딩(`PAD_IDX`)을 구분하여,
모델이 패딩 토큰을 연산에 포함하지 않도록 함.

메모리(encoder 출력) 생성

- 원문(소스 문장)을 토큰 인덱스로 변환해 `[S_src, 1]` 형태의 `source_tensor` 생성.
- `src_mask` 를 `[S_src, S_src]` 크기의
  모두 False boolean 행렬로 설정.
- `model.encode(source_tensor, src_mask)` 호출 →
  마지막 인코더 블록의 출력을 메모리(memory)로 반환

디코딩 루프

- 디코더 입력 시퀀스 `ys` 초기화
- 최대 길이 `max_len = num_tokens + 5` 만큼 반복
  → 그리디 디코딩 : 매 스텝 가장 높은 확률 토큰을 선택하므로 빠르나,
  전역 최적을 보장하지 않음.
- 반복 종료 후 ys에는 `[BOS, y1, y2, …, yT, EOS]`
  인덱스 시퀀스가 저장됨.

토큰 → 문장 반환

- `ys.flatten()` → NumPy 배열
- 어휘사전(`vocab_transform[TGT_LANGUAGE]`)의 `lookup_tokens(...)` 로 토큰 문자열 리스트 변환
- 슬라이싱 [1:-1]로 `<bos>` · `<eos>` 제거
- 공백 `' '` 로 join → 최종 번역문 반환

GPT (Generative Pre-trained Transformer)

- Transformer 디코더만을 여러 겹 쌓아 만든 언어 생성 모델

- 대규모 텍스트 말뭉치로 사전 학습(pre-training)한 후,
  특정 자연어 처리 작업에 맞게 미세 조정(fine-tuning).

- 자연어 생성·기계 번역·질의응답·요약 등 다양한 작업에서 뛰어난 성능

주요 버전 및 매개변수 규모

모델	발표 연도	매개변수 수	특징
GPT-1	2018	약 1.2억	최초 제안, Transformer 디코더 활용
GPT-2	2019	약 15억	규모 대폭 확대, 더 강력한 생성 능력
GPT-3	2020	약 1,750억	초대형 모델, 뛰어난 문장 생성 성능
GPT-3.5	2022	–	**RLHF**(Human Feedback) 도입 → InstructGPT
GPT-4	2023	공개 미정	텍스트·이미지 멀티모달 학습 지원

RLHF (Reinforcement Learning from Human Feedback)

- 인간 평가자가 모델 출력을 평가
- 좋은 결과에 보상(reward) → 정책(policy)을 강화 학습
- 사전 학습만 사용할 때보다 정확도·안정성 크게 향상

GPT-1

모델 구조

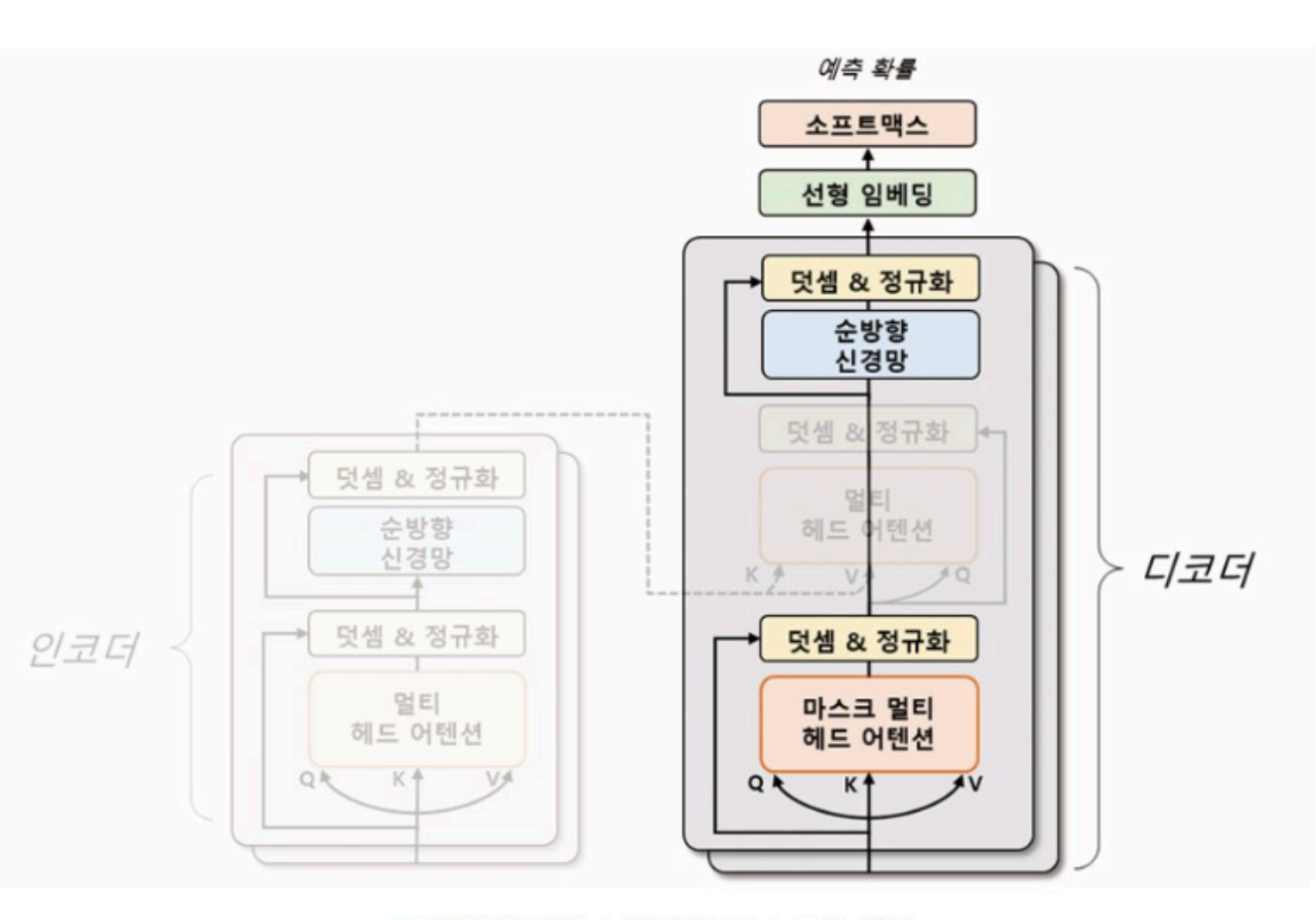


그림 7.7 트랜스포머와 GPT–1 모델 비교

- Transformer의 디코더(Decoder) 블록만 12개 쌓음.

- 인코더-디코더 어텐션 없이,
  디코더 내부의 마스크드 멀티헤드 셀프 어텐션만 사용.

- 단방향(Unidirectional) : 이전 토큰들만 보고 다음 토큰을 예측.

입력 처리

- 각 토큰에 대해 토큰 임베딩 + 위치(Positional) 임베딩 합산
- 문장 앞에 `<start>` 토큰, 마지막에 `<extract>` 토큰 등 특수 토큰 삽입

사전 학습(Pre-training) 작업

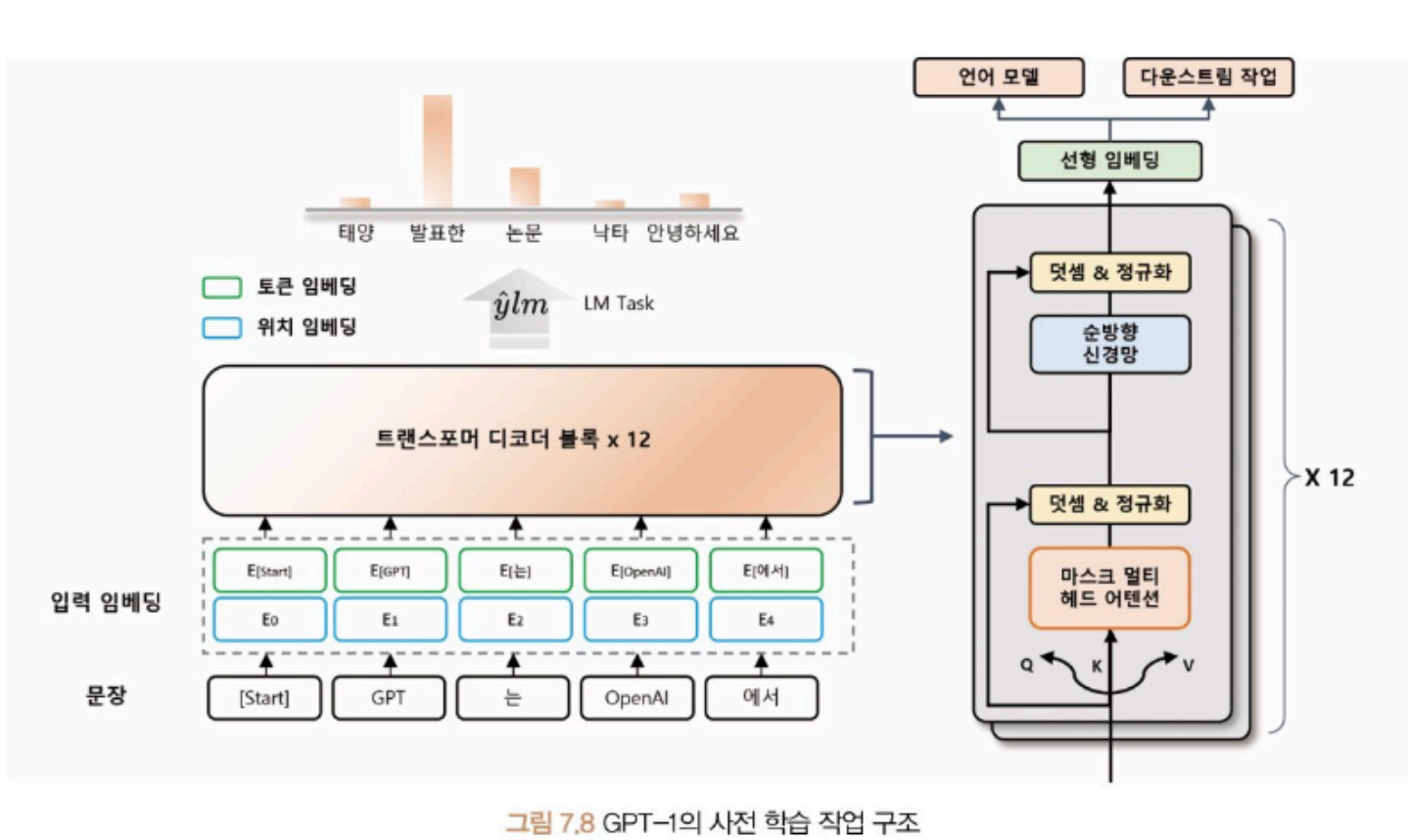


그림 7.8 GPT–1의 사전 학습 작업 구조

- BookCorpus (약 4.5 GB 분량, 7,000여 권 미출판 도서)로
  언어 모델링(다음 토큰 예측)
- 블록 단위로 일정 길이만 보고 다음 토큰만 맞추는 **비지도 학습**

마스크 처리

- 셀프 어텐션 마스크 : 미래(오른쪽) 토큰 정보를 가리도록 `-inf` 채우기
  → 현재 위치까지의 문맥만 활용해 예측

보조 학습(Auxiliary Learning)

- 언어 모델링 외에도, 다운스트림 작업에 맞춘 추가 손실 함수를 함께 학습

다운스트림(Down-stream) 작업 구조

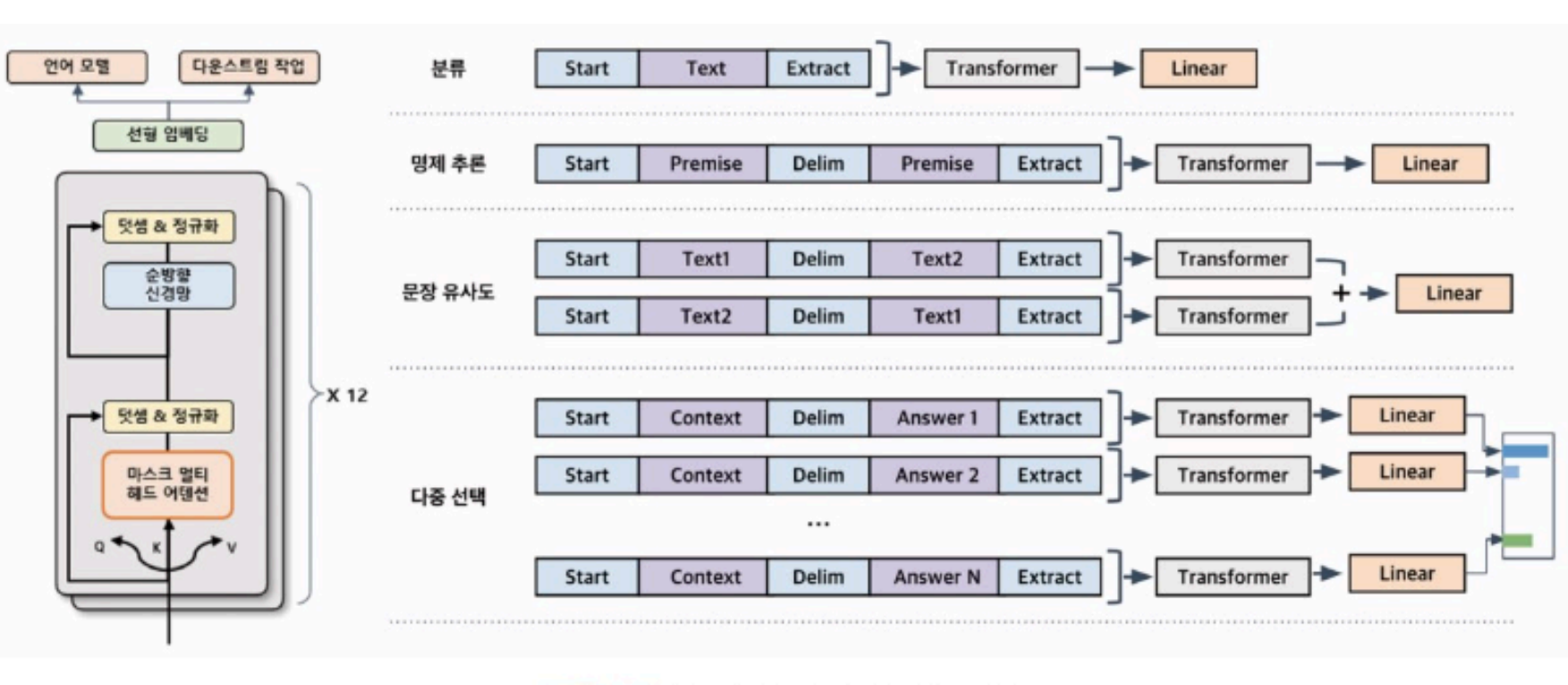

언어 모델
다운스트림 작업
선형 임베딩
덧셈 & 정규화
순방향 신경망
덧셈 & 정규화
마스크 멀티 헤드 어텐션
Q
K
V
X 12
분류
Start
Text
Extract
Transformer
Linear
명제 추론
Start
Premise
Delim
Premise
Extract
Transformer
Linear
문장 유사도
Start
Text1
Delim
Text2
Extract
Transformer
Start
Text2
Delim
Text1
Extract
Transformer
+
Linear
다중 선택
Start
Context
Delim
Answer 1
Extract
Transformer
Linear
Start
Context
Delim
Answer 2
Extract
Transformer
Linear
...
Start
Context
Delim
Answer N
Extract
Transformer
Linear

그림 7.9 GPT–1의 다운스트림 작업 구조

- Start–Text–Extract 형태의 입력을 넣으면,
- Transformer→Linear 로 각 작업별 출력(logit)을 생성
- 작업마다 별도의 Extract 토큰 위치의 logits을 손실 계산에 사용

GPT-2

모델 규모 및 구조

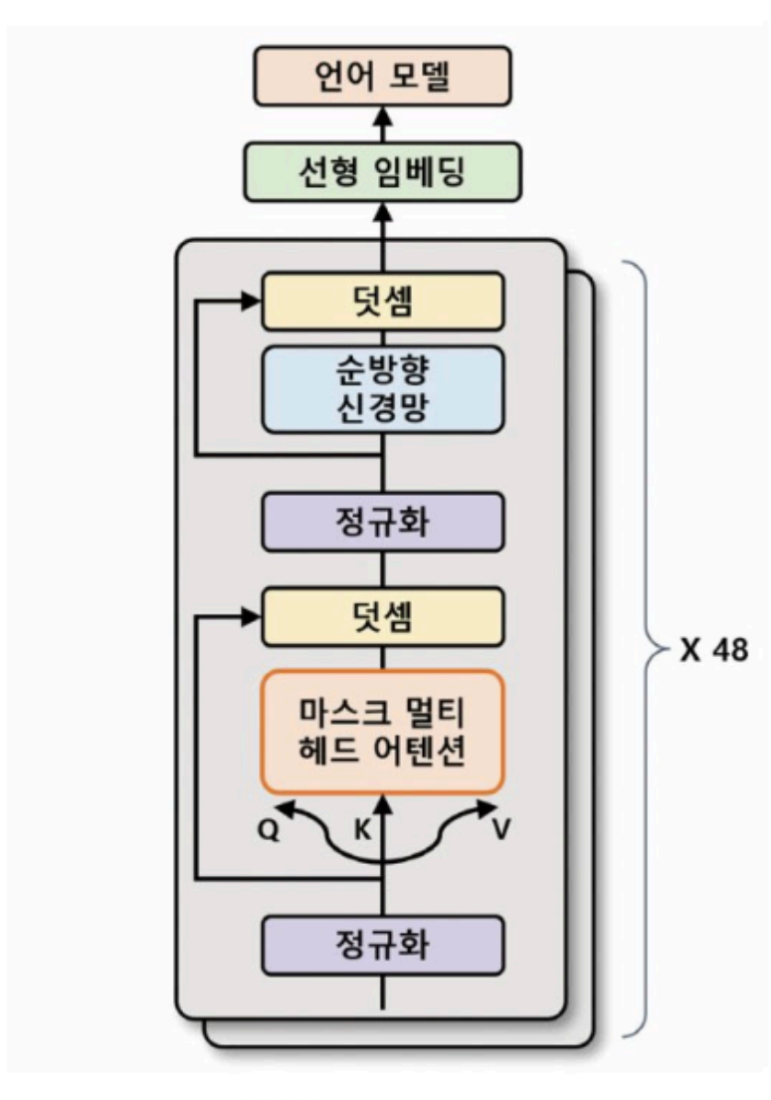

언어 모델
선형 임베딩
덧셈
순방향 신경망
정규화
덧셈
마스크 멀티 헤드 어텐션
Q
K
V
정규화
X 48

그림 7.10 GPT–2의 모델 구조

- 블록 수 : 48개의 디코더 블록 (Decoder-only Transformer)
- 입력 길이 : 최대 1,024 토큰까지 처리 (GPT-1 대비 512→1,024 확장)
- 매개변수 수 : 약 15억 개

입력 및 어텐션

- 단방향(Self-Attention, 마스크드 멀티헤드 어텐션)
  : 미래 토큰 정보는 볼 수 없도록 마스크 처리,
  이전 토큰들만 활용해 다음 토큰 예측.

- 토큰 임베딩 + 위치 인코딩 → 디코더 첫 블록 입력

사전 학습(Pre-trained) 데이터

- 웹 크롤링 문서: 약 8백만 개 문서, 총 40 GB
- 언어 모델링(다음 토큰 예측) 작업

제로샷(Zero-Shot) 활용

- 사전 학습만으로 직접 다운스트림 태스크(ex. 번역, 요약) 수행 가능
- 별도의 미세 조정 없이 “문장 A = 문장 B” 형태로 학습
  → 다양한 작업에 적용

GPT-3

모델 구조

- GPT-2와 거의 동일한 디코더 전용 Transformer 구조
- 96개의 멀티-헤드 어텐션 헤드
- 96개의 디코더 블록 층

크기 및 용량

- 토큰 임베딩 차원: 1,600 → 12,288로 대폭 확대
- 최대 컨텍스트 길이: 1,024 → 2,048 토큰
- 매개변수 수: GPT-2 대비 약 116배↑ (1,750억 개 규모)

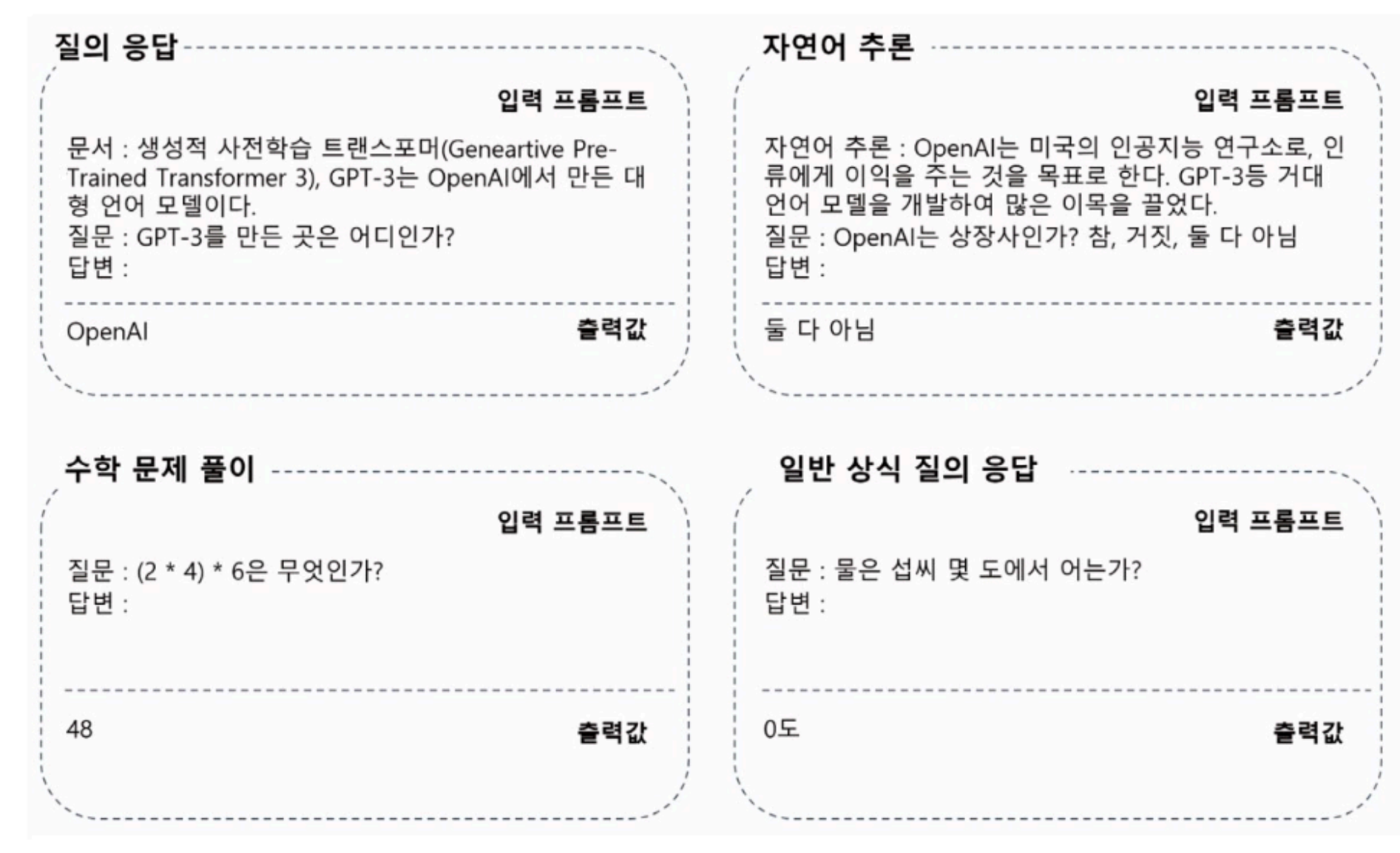
어텐션 최적화

연산량 절감을 위해 Sparse Attention(희소 어텐션)과 일반 어텐션 혼합 사용

사전 학습 데이터

- 웹 크롤링, 위키백과, 전자책 등에서 수집한 약 45TB 분량
- 다음 토큰 예측 언어 모델링(task)으로만 학습
- 다운스트림 태스크용 추가 미세조정 없이도
  (zero-/few-/one-shot) 다양한 작업 수행

Prompt-based 활용 예


질의 응답
입력 프롬프트
출력값
자연어 추론
입력 프롬프트
출력값
수학 문제 풀이
입력 프롬프트
출력값
일반 상식 질의 응답
입력 프롬프트
출력값

문서 : 생성적 사전학습 트랜스포머(Geneartive Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다.
질문 : GPT-3를 만든 곳은 어디인가?
답변 :

OpenAI

자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다.
질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님
답변 :

둘 다 아님

질문 : (2 * 4) * 6은 무엇인가?
답변 :

48

질문 : 물은 섭씨 몇 도에서 어는가?
답변 :

0도

그림 7.11 GPT–3의 다운스트림 작업 프롬프트 예시

장·단점

장점

- 거대한 규모로 제로샷/원샷/소수샷 대응 성능 우수
- 자연어 이해·생성, 수학·추론, 번역·요약 등 다양한 분야에서
  뛰어난 일반화 능력

단점

- 학습·추론 비용이 매우 높아 일반 사용자나 소규모 환경에서 활용 어려움
- 데이터 편향이나 부정확한 정보 생성 위험 → 검증 과정 필수
- 인간적 상식·윤리 판단 수준은 여전히 제한적

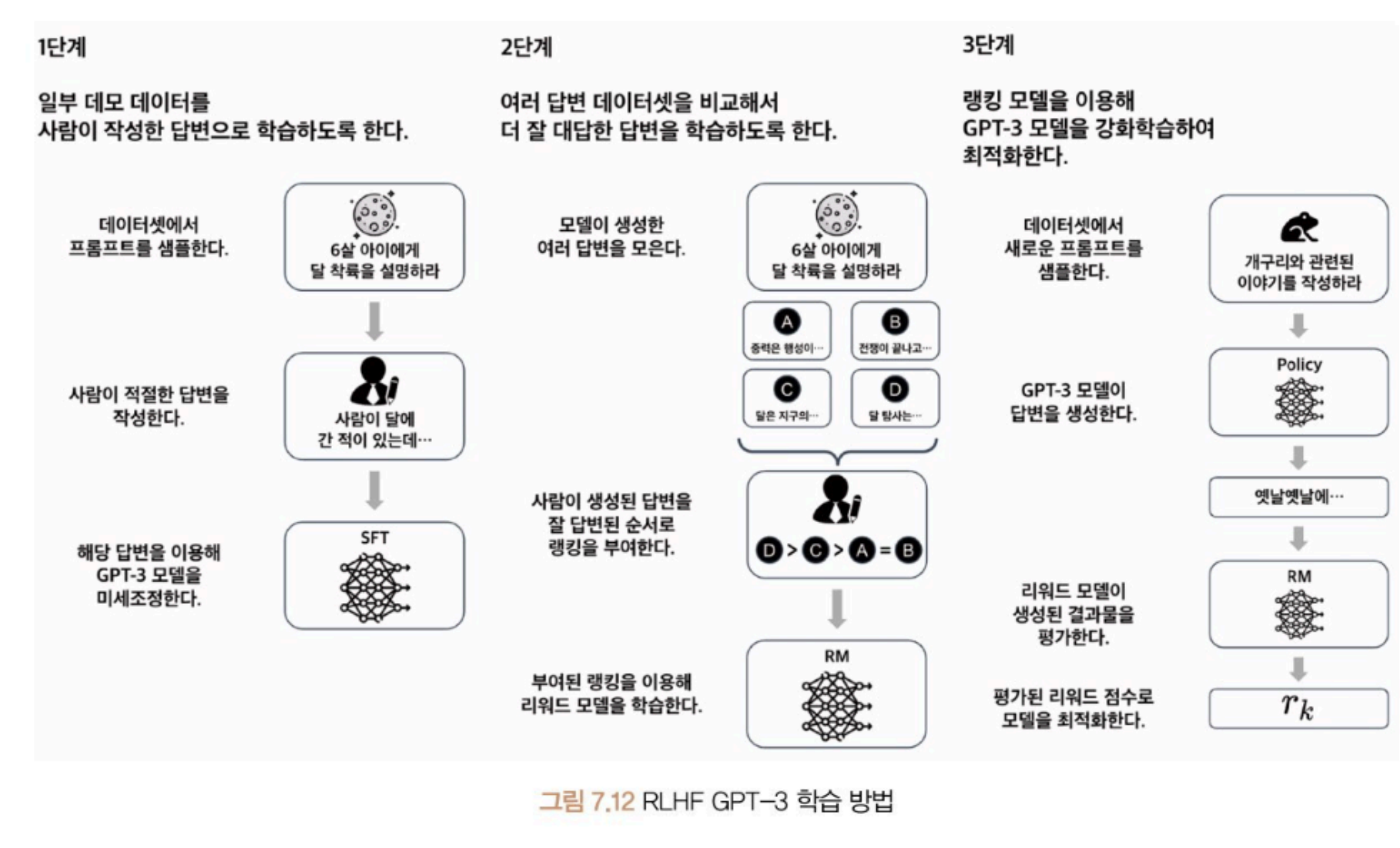
GPT 3.5 (InstructGPT)

- 기본 구조 : GPT-3의 디코더 전용 Transformer 아키텍처 계승

- 추가 학습 기법 : RLHF(Reinforcement Learning from Human Feedback) 도입

- 목표 : 모델이 더 자연스럽고 유용한 답변을 생성하도록 보강

RLHF 학습 단계


1단계
2단계
3단계
6살 아이에게 달 착륙을 설명하라
사람이 달에 간 적이 있는데…
SFT
6살 아이에게 달 착륙을 설명하라
A
B
C
D
D > C > A = B
RM
개구리와 관련된 이야기를 작성하라
Policy
옛날옛날에…
RM
r_k

1단계
일부 데모 데이터를 사람이 작성한 답변으로 학습하도록 한다.
데이터셋에서 프롬프트를 샘플한다.
사람이 적절한 답변을 작성한다.
해당 답변을 이용해 GPT-3 모델을 미세조정한다.

2단계
여러 답변 데이터셋을 비교해서 더 잘 대답한 답변을 학습하도록 한다.
모델이 생성한 여러 답변을 모은다.
사람이 생성된 답변을 잘 답변된 순서로 랭킹을 부여한다.
부여된 랭킹을 이용해 리워드 모델을 학습한다.

3단계
랭킹 모델을 이용해 GPT-3 모델을 강화학습하여 최적화한다.
데이터셋에서 새로운 프롬프트를 샘플한다.
GPT-3 모델이 답변을 생성한다.
리워드 모델이 생성된 결과물을 평가한다.
평가된 리워드 점수로 모델을 최적화한다.

그림 7.12 RLHF GPT–3 학습 방법

Supervised Fine-Tuning (SFT)

- 일부 프롬프트에 대해 사람이 작성한 ‘정답’ 문장을 수집
- 이 데이터를 이용해 GPT-3 모델을 지도 학습 (미세조정)

Reward 모델 학습

- GPT-3가 생성한 여러 후보 답변을 모아, 사람이 선호도 순으로 랭킹
- 이 랭킹을 학습 신호로 사용해 보상 모델(Reward Model) 학습

최종 강화학습 (Policy Optimization)

- SFT로 미세조정된 모델(Policy)을 고정.
- 프롬프트에 대해 Policy가 생성한 답변을
  보상 모델이 평가한 점수로 강화학습.
- PPO(Proximal Policy Optimization) 등 알고리즘으로
  Policy 파라미터 업데이트.

한계 및 유의점

환각(Hallucination)

- 현실 검증되지 않은 정보 생성 가능성이 아직 존재
- 언어 모델이 추론과 판단 능력을 갖추지 못해 부정확한 답 제시

GPT-4

멀티모달 처리

- 텍스트뿐 아니라 이미지 입력도 이해·처리 가능
- 이전 세대(순수 텍스트) 대비 활용 범위 확장

늘어난 컨텍스트 길이

- 최대 32,768 토큰(약 25,000단어)까지 한 번에 처리
- GPT-3.5(최대 4,096토큰) 대비 약 8배 긴 문맥 유지

학습 성능 향상

MMLU(BigBench 계열) 벤치마크

- GPT-3.5: ≈70% → GPT-4: ≈85%
- 한국어 번역판 MMLU에서도 77% 기록(GPT-3.5 대비↑)

모델 실습

: GPT-2를 이용한 문장 분류(Sequence Classification)

GPT-2 모델 구조

주요 구성 요소

- `wte` (token embedding)
- `wpe` (position embedding)
- `drop` (dropout)
- `h` (12개 디코더 블록)
  : 각 블록에 LayerNorm, causal self-attention, MLP
- `ln_f` (마지막 LayerNorm)
- `lm_head` (출력 로짓을 만드는 linear layer)

문장 생성(pipeline)

: `pipeline(task="text-generation", model="gpt2")` 로
생성 파이프라인 인스턴스 생성

주요 파라미터

- `text_inputs` : 프롬프트(입력 문장)
- `max_length` : 생성할 최대 토큰 길이
- `num_return_sequences` : 출력 시퀀스 개수
- `pad_token_id` : 자유 생성(open-end)을 위해
  패딩 토큰으로 사용할 ID(eos 토큰 사용)

CoLA 데이터셋 로드 및 전처리

`torchtext.datasets.CoLA`로 train/dev/test 분할 불러오기.

`AutoTokenizer.from_pretrained("gpt2")` 사용,
GPT-2는 pad 토큰이 없으므로
`tokenizer.pad_token = tokenizer.eos_token` 으로 설정

`collator` 함수

- 배치 내 (source, label, text) 묶음 → 텍스트만 추출
- `tokenizer(..., padding="longest", truncation=True, return_tensors="pt")` 로 토크나이즈
- `input_ids`, `attention_mask`, `labels` 텐서 생성 및 디바이스로 이동

모델 초기화

패딩 토큰 설정
: GPT-2 토크나이저에는 원래 pad 토큰이 없으므로

```
model.config.pad_token_id = model.config.eos_token_id
```

로 EOS 토큰을 패딩 토큰으로 대체

학습 결과

에폭	Train Loss	Val Loss	Val Accuracy
1	0.6048	0.5266	0.7238
2	0.4834	0.4927	0.7673
3	0.3535	0.5156	0.7900

CoLA 데이터(8,550개 학습 샘플)만으로도 약 74% 정확도를 달성

chelseey

이전 포스트
Neural Networks I

다음 포스트
Neural Networks II

0개의 댓글

댓글을 작성하세요

댓글 작성

관심 있을 만한 포스트

Powered by Stellate